Python 程式語言

Lecture 4: Modular and Functional Programming

Instructor: 顏家珩 Chia-Heng Yen

Course Roadmap

- Fundamentals & Environment
  - Introduction to Python and Development Environment Setup
  - Basic Syntax and Flow Control
  - Data Structures and Collections
- Advanced Programming
  - Modular and Functional Programming
  - File Handling and Exception Handling
  - Object-Oriented Programming
  - Algorithm Implementation in Python
  - Modules and Package Applications
- User Interface Development
  - GUI Programming with Tkinter and PyQt
- Data Processing & Analysis
  - Data Analysis Fundamentals
  - Scientific Computing Applications
  - Data Visualization
- AI & ML Applications
  - Introduction to Machine Learning with Applications
  - AI-Related Library in Python Applications

Outline

- Understand modular programming concepts
- Master function definition and usage
- Apply functional programming principles
- Create custom modules and packages
- Implement iterators and generators

What is a Function?

- A function is a reusable block of code that performs a specific task
- Benefits
  - Code Reusability
  - Modularity
  - Easier Testing
  - Better Organization
  - Reduced Complexity

```
# Without functions
print("Hello, Alice!")
print("Welcome to our program!")
print("Have a great day!")

print("Hello, Bob!")
print("Welcome to our program!")
print("Have a great day!")

# With functions
def greet_user(name):
        print(f"Hello, {name}!")
        print("Welcome to our program!")
        print("Have a great day!")

greet_user("Alice")
greet_user("Bob")
```

Function Definition and Call

- Basic Syntax

```
def function_name(parameters):
    """
    Docstring: Function description
    """
    # Function body
    statements
    return value # Optional
```

- Simple Examples

```
# Function without parameters
def say_hello():
    """Print a greeting message"""
    print("Hello, World!")

# Function with parameters
def greet(name, age):
    """Greet a person with name and age""“
    print(f"Hello {name}, you are {age} years old!")

# Function with return value
def add_numbers(a, b):
    """Add two numbers and return the result"""
    result = a + b
    return result

# Function calls
say_hello() # Output: Hello, World!
greet("Alice", 25) # Output: Hello Alice, you are 25 years old!
sum_result = add_numbers(5, 3) # sum_result = 8
print(sum_result) # Output: 8
```

Parameter Types

- Positional Parameters

```
def introduce(name, age, city):
    """Introduce a person with positional parameters"""
    print(f"Hi, I'm {name}, {age} years old, from {city}")

# Must provide arguments in correct order
introduce(“Alice”, 25, “Taipei”)  # introduce(25, "Alice", "Taipei") # Wrong order!
```

- Keyword Parameters

```
def create_profile(name, age, city, occupation):
    """Create user profile using keyword arguments"""
    print(f"Profile: {name}, {age}, {city}, {occupation}")
# Can use keywords in any order
create_profile(name="Bob", city="Kaohsiung", age=30, occupation="Engineer")
create_profile(age=28, name="Carol", occupation="Designer", city="Taichung")

# Mix positional and keyword (positional first!)
create_profile("David", 35, city="Tainan", occupation="Teacher")
```

Parameter Types

- Default Parameters

```
def greet_user(name, greeting="Hello", punctuation="!"):
        """Greet user with default greeting and punctuation"""
        message = f"{greeting}, {name}{punctuation}"
        print(message)

# Using defaults
greet_user("Alice") # Hello, Alice!
greet_user("Bob", "Hi") # Hi, Bob!
greet_user("Carol", "Hey", ".") # Hey, Carol.
greet_user("David", punctuation="?") # Hello, David?
```

```
# Important: Default values are evaluated once!
def add_to_list(item, target_list=[]): # Dangerous!
        target_list.append(item)
        return target_list

print(add_to_list(1)) # [1]
print(add_to_list(2)) # [1, 2] ← ?
print(add_to_list(3)) # [1, 2, 3] ← ! ?

# Better approach
def add_to_list_safe(item, target_list=None):
        if target_list is None:
                target_list = []
        target_list.append(item)
        return target_list
```

Parameter Types

- Variable-Length Parameters
  - *args - Variable Positional Arguments

```
def calculate_sum(*numbers):
        """Calculate sum of any number of arguments"""
        print(f"Received arguments: {numbers}") # Tuple of arguments
        total = sum(numbers)
        return total

# Examples
print(calculate_sum(1, 2, 3)) # 6
print(calculate_sum(1, 2, 3, 4, 5)) # 15
print(calculate_sum(10)) # 10
print(calculate_sum()) # 0
```

```
# Practical example
def print_info(name, *hobbies):
        """Print person's name and hobbies"""
        print(f"Name: {name}")
        if hobbies:
                print("Hobbies:", ", ".join(hobbies))
        else:
                print("No hobbies listed")

print_info("Alice", "reading", "swimming", "coding")
print_info("Bob")
```

Parameter Types

- Variable-Length Parameters
  - **kwargs - Variable Keyword Arguments

```
def create_user_profile(name, **details):
        """Create user profile with flexible details"""
        print(f"User: {name}")
        print("Details:")
        for key, value in details.items():
                print(f" {key}: {value}")


# Examples
create_user_profile("Alice", age=25, city="Taipei", job="Engineer")
create_user_profile("Bob", age=30, country="Taiwan", language="Python")
```

```
# Combining all parameter types
def complex_function(required, default_param="default", *args, **kwargs):
        """Function with all parameter types"""
        print(f"Required: {required}")
        print(f"Default: {default_param}")
        print(f"Args: {args}")
        print(f"Kwargs: {kwargs}")
complex_function("must_have", "custom", 1, 2, 3, name="Alice", age=25)
# Required: must_have
# Default: custom
# Args: (1, 2, 3)
# Kwargs: {'name': 'Alice', 'age': 25}
```

- *args will always be a tuple, **kwargs will always be a dict

Variable Scope

- Local vs Global Variables

```
# Global variable
global_var = "I'm global"

def test_scope():
    # Local variable
    local_var = "I'm local"
    print(f"Inside function - Global: {global_var}")
    print(f"Inside function - Local: {local_var}")

test_scope()
print(f"Outside function - Global: {global_var}")
# print(local_var) # NameError! local_var is not accessible
```

```
# Modifying global variables
counter = 0 # Global
def increment():
    global counter # Must declare global to modify
    counter += 1
    print(f"Counter is now: {counter}")

increment() # Counter is now: 1
increment() # Counter is now: 2
print(f"Final counter: {counter}") # Final counter: 2
```

LEGB Rule

- Local → Enclosing → Global → Built-in

```
# Built-in scope 內建範圍
# len, print, max, min, etc.

# Global scope 全域範圍
global_name = "Global Alice"

def outer_function():
        # Enclosing scope 封閉範圍
        enclosing_name = "Enclosing Bob"

        def inner_function():
                # Local scope 區域範圍
                local_name = "Local Carol"
                print(f"Local: {local_name}") # Local Carol
                print(f"Enclosing: {enclosing_name}") # Enclosing Bob
                print(f"Global: {global_name}") # Global Alice
                print(f"Built-in: {len('hello')}") # Built-in: 5
        inner_function()
outer_function()
```

```
x = "Global" # Global scope
def outer():
        x = "Enclosing" # Enclosing scope
        def inner():
                x = "Local" # Local scope
                print(x) # Output ?
        inner()
outer() # Output : Local
```

Modifying Outer Variables: global and nonlocal

- Problem: Direct assignment creates a new variable

```
count = 0
def increment():
        count = count + 1 #  Error! # Python thinks you want to create a new local variable count
increment() # UnboundLocalError
```

```
# Solution 1: Using global
count = 0
def increment():
        global count
        # Declare that we're using the global variable
        count = count + 1
increment() # 1
```

```
# Solution 2: Using nonlocal (for Enclosing scope)
def outer():
        count = 0
        def increment():
                nonlocal count
                # Declare that we're using the enclosing function's variable
                count += 1
                print(f"Inner count: {count}")
                increment() # Inner count: 1
                increment() # Inner count: 2
                print(f"Outer count: {count}") # Outer count: 2
outer()
```

Return Statement

- Basic Return

```
def square(number):
    """Return the square of a number"""
    return number ** 2
result = square(5)
print(result) # 25

# Function without explicit return
def greet(name):
    print(f"Hello, {name}")
    # Implicitly returns None

result = greet("Alice")
print(result) # None
```

- Multiple Return Values

```
def get_name_parts(full_name):
    """Split full name into first and last name"""
    parts = full_name.split()
    if len(parts) >= 2:
        return parts[0], parts[-1] # Returns a tuple
    else:
        return parts[0], ""

# Unpacking returned values
first, last = get_name_parts("Alice Johnson")
print(f"First: {first}, Last: {last}") # First: Alice, Last: Johnson

# Can also receive as tuple
name_parts = get_name_parts("Bob Smith")
print(name_parts) # ('Bob', 'Smith')
```

Return Statement

- Early Return

```
def check_age(age):
        """Check if age is valid for voting"""
        if age < 0:
                return "Invalid age"
        if age < 18:
                return "Too young to vote"
        if age > 120:
                return "Age seems unrealistic"
        return "Eligible to vote"

# Examples
print(check_age(25)) # Eligible to vote
print(check_age(16)) # Too young to vote
print(check_age(-5)) # Invalid age
```

Functions as First-Class Objects (一等函式)

- Functions are Objects
  - In Python, functions are first-class objects, meaning they can be:
    - Assigned to variables
    - Passed as arguments
    - Returned from other functions
    - Stored in data structures
  - Key Points
    - say_hello = greet
      - Copies the function reference (not calling it)
    - say_hello = greet()
      - Calls the function, assigns the return value

```
def greet(name):
    return f"Hello, {name}!"

# Function assigned to variable
say_hello = greet
print(say_hello("Alice")) # Hello, Alice!

# Function as dictionary value
operations = {
    'add': lambda x, y: x + y,
    'subtract': lambda x, y: x - y,
    'multiply': lambda x, y: x * y,
    'divide': lambda x, y: x / y if y != 0 else "Cannot divide by zero"
}

print(operations['add'](5, 3)) # 8
print(operations['multiply'](4, 7)) # 28
```

Higher-Order Functions

- Functions that take functions as parameters

```
def apply_operation(numbers, operation):
    """Apply an operation to all numbers in a list"""
    result = []
    for num in numbers:
        result.append(operation(num))
    return result

# Define some operations
def square(x):
    return x ** 2
def cube(x):
    return x ** 3
def double(x):
    return x * 2
```

```
# Use higher-order function
numbers = [1, 2, 3, 4, 5]
print(apply_operation(numbers, square))
# [1, 4, 9, 16, 25]
print(apply_operation(numbers, cube))
# [1, 8, 27, 64, 125]
print(apply_operation(numbers, double))
# [2, 4, 6, 8, 10]
```

Higher-Order Functions

- Functions that return functions

```
def create_multiplier(factor):
    """Create a function that multiplies by a specific factor"""
    def multiplier(number):
        return number * factor
    return multiplier

# Create specific multiplier functions
double = create_multiplier(2)
triple = create_multiplier(3)
times_ten = create_multiplier(10)

print(double(5)) # 10
print(triple(4)) # 12
print(times_ten(7)) # 70
```

```
# More practical example
def create_validator(min_length, max_length):
    """Create a password validation function"""
    def validate_password(password):
        if len(password) < min_length:
            return f"Password too short (minimum {min_length} characters)"
        elif len(password) > max_length:
            return f"Password too long (maximum {max_length} characters)" else:
            return "Password length is valid"
    return validate_password
# Create different validators
basic_validator = create_validator(6, 20)
strict_validator = create_validator(12, 50)

print(basic_validator("hello")) # Password too short (minimum 6 characters)
print(basic_validator("hello123")) # Password length is valid
print(strict_validator("hello123")) # Password too short (minimum 12 characters)
```

Lambda Functions

- Anonymous (匿名函式) Functions

```
# Traditional function
def add(x, y):
        return x + y
# Lambda equivalent
add_lambda = lambda x, y: x + y

print(add(5, 3)) # 8
print(add_lambda(5, 3)) # 8

# Lambda with single parameter
square = lambda x: x ** 2
print(square(4)) # 16

# Lambda in list comprehension
numbers = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x ** 2, numbers))
print(squared) # [1, 4, 9, 16, 25]
```

Lambda Functions

- Common Lambda Use Cases

```
# Sorting with custom key
students = [
        {'name': 'Alice', 'age': 23, 'grade': 88},
        {'name': 'Bob', 'age': 21, 'grade': 92},
        {'name': 'Carol', 'age': 22, 'grade': 85}
]

# Sort by age
students_by_age = sorted(students, key=lambda student: student['age'])
print(students_by_age)

# Sort by grade (descending)
students_by_grade = sorted(students, key=lambda student: student['grade'], reverse=True)
print(students_by_grade)
```

```
# Filtering with lambda
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
 even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(even_numbers) # [2, 4, 6, 8, 10]

# Dictionary operations with lambda
grades = {'Alice': 85, 'Bob': 92, 'Carol': 78, 'David': 95}
top_students = dict(filter(lambda item: item[1] >= 90, grades.items()))
print(top_students) # {'Bob': 92, 'David': 95}
```

Built-in Higher-Order Functions

- map() Function

```
# Apply function to all items in iterable
numbers = [1, 2, 3, 4, 5]

# Using map with built-in function
squared = list(map(lambda x: x ** 2, numbers))
print(squared) # [1, 4, 9, 16, 25]

# Converting strings to integers
str_numbers = ['1', '2', '3', '4', '5']
int_numbers = list(map(int, str_numbers))
print(int_numbers) # [1, 2, 3, 4, 5]
```

```
# Multiple iterables
list1 = [1, 2, 3]
list2 = [4, 5, 6]
sums = list(map(lambda x, y: x + y, list1, list2))
print(sums) # [5, 7, 9]

# Practical example: Temperature conversion
celsius_temps = [0, 20, 30, 40, 100]
fahrenheit_temps = list(map(lambda c: c * 9/5 + 32,
celsius_temps))
print(fahrenheit_temps) # [32.0, 68.0, 86.0, 104.0, 212.0]
```

Built-in Higher-Order Functions

- filter() Function

```
# Filter items based on condition
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Filter even numbers
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(even_numbers) # [2, 4, 6, 8, 10]

# Filter numbers greater than 5
greater_than_five = list(filter(lambda x: x > 5, numbers))
print(greater_than_five) # [6, 7, 8, 9, 10]
```

```
# Filter strings by length
words = ['cat', 'elephant', 'dog', 'hippopotamus', 'bird']
long_words = list(filter(lambda word: len(word) > 5, words))
print(long_words) # ['elephant', 'hippopotamus']

# Filter None values
mixed_list = [1, None, 2, '', 3, 0, 4, None]
filtered_list = list(filter(None, mixed_list)) # None as filter function
print(filtered_list) # [1, 2, 3, 4]
```

Built-in Higher-Order Functions

- reduce() Function

```
from functools import reduce

# Reduce list to single value
numbers = [1, 2, 3, 4, 5]

# Sum all numbers
total = reduce(lambda x, y: x + y, numbers)
print(total) # 15

# Find maximum
maximum = reduce(lambda x, y: x if x > y else y, numbers)
print(maximum) # 5

# Calculate factorial
factorial_5 = reduce(lambda x, y: x * y, range(1, 6))
print(factorial_5) # 120
```

```
# String concatenation
words = ['Hello', 'World', 'Python', 'Programming']
sentence = reduce(lambda x, y: x + ' ' + y, words)
print(sentence) # Hello World Python Programming

# With initial value
numbers = [1, 2, 3, 4, 5]
sum_with_initial = reduce(lambda x, y: x + y, numbers, 100)
print(sum_with_initial) # 115 (100 + 15)
```

Recursive Functions

- Basic Recursion Concept
  - A recursive function calls itself to solve a smaller version of the same problem

```
# Classic example: Factorial (階層)
def factorial(n):
    """Calculate factorial using recursion"""
    # Base case
    if n == 0 or n == 1: return 1

    # Recursive case
    return n * factorial(n - 1)

print(factorial(5)) # 120 (5 * 4 * 3 * 2 * 1)
print(factorial(0)) # 1
```

```
# Fibonacci sequence
def fibonacci(n):
    """Calculate nth Fibonacci number"""
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

# Generate first 10 Fibonacci numbers
for i in range(10):
    print(f"F({i}) = {fibonacci(i)}")
```

Recursive Functions

- More Practical Examples

```
# Directory tree traversal (conceptual)
def count_files(directory_structure):
    """Count total files in nested directory structure"""
    count = 0
    for item in directory_structure:
        if isinstance(item, list): # Subdirectory
            count += count_files(item)
        else: # File
            count += 1
    return count

# Example directory structure
directories = [
    'file1.txt',
    'file2.py',
    ['subdir1', 'file3.txt', 'file4.py'],
    ['subdir2', 'file5.txt', ['subdir3', 'file6.py']]
]

print(count_files(directories)) # 6
```

```
# Binary search (recursive)
def binary_search(arr, target, left=0, right=None):
    """Recursive binary search"""
    if right is None:
        right = len(arr) - 1
    if left > right:
        return -1 # Not found
    mid = (left + right) // 2

    if arr[mid] == target:
        return mid
    elif arr[mid] > target:
        return binary_search(arr, target, left, mid - 1)
    else:
        return binary_search(arr, target, mid + 1, right)

# Test binary search
sorted_numbers = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
print(binary_search(sorted_numbers, 7)) # 3 (index of 7)
print(binary_search(sorted_numbers, 12)) # -1 (not found)
```

What are Modules (模組)?

- A module is a file containing Python code (functions, classes, variables)
- Benefits
  - Code Organization
  - Code Reusability
  - Namespace Management
  - Collaboration

What are Modules?

```
# Example: math_utils.py (saved as separate file)
""" Mathematical utility functions """

PI = 3.14159265359

def circle_area(radius):
    """Calculate area of a circle"""
    return PI * radius ** 2
def circle_circumference(radius):
    """Calculate circumference of a circle"""
    return 2 * PI * radius
def is_prime(n):
    """Check if a number is prime"""
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True
```

```
def factorial(n):
    """Calculate factorial of a number"""
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

What are Modules?

- Import Statements

```
# 1. Import entire module
import math_utils

# Usage
print(math_utils.PI) # 3.14159265359
print(math_utils.circle_area(5)) # 78.53981633975
print(math_utils.is_prime(17)) # True

# 2. Import specific functions
from math_utils import circle_area, is_prime

# Usage (no module prefix needed)
print(circle_area(3)) # 28.274333882308138
print(is_prime(15)) # False
```

```
# 3. Import with alias
import math_utils as math_tools

# Usage
print(math_tools.factorial(5)) # 120

# 4. Import all (not recommended!)
from math_utils import *

# Usage
print(PI) # 3.14159265359
print(circle_circumference(4)) # 25.132741228718347
```

Standard Library Modules

- Commonly Used Modules

```python
# 1. math module
import math
print(math.pi) # 3.141592653589793
print(math.sqrt(16)) # 4.0
print(math.ceil(4.2)) # 5
print(math.floor(4.8)) # 4
print(math.factorial(5)) # 120

# 2. random module
import random

print(random.randint(1, 10)) # Random integer between 1-10
print(random.choice(['a', 'b', 'c'])) # Random choice from list
print(random.random()) # Random float between 0-1

numbers = [1, 2, 3, 4, 5]
random.shuffle(numbers)
print(numbers) # Shuffled list
```

```python
# 3. datetime module
from datetime import datetime, date, timedelta

now = datetime.now()
print(f"Current time: {now}")

today = date.today()
print(f"Today: {today}")

tomorrow = today + timedelta(days=1)
print(f"Tomorrow: {tomorrow}")

# 4. os module
import os

print(f"Current directory: {os.getcwd()}")
print(f"User home: {os.path.expanduser('~')}")
# print(os.listdir('.')) # List current directory contents
```

Creating Custom Modules

- File Structure

```
project/
├── main.py
├── calculator.py
└── string_utils.py
```

- calculator.py

```python
""" Calculator module with basic operations """

def add(a, b):
    """Add two numbers"""
    return a + b
def subtract(a, b):
    """Subtract two numbers"""
    return a - b
def multiply(a, b):
    """Multiply two numbers"""
    return a * b
def divide(a, b):
    """Divide two numbers"""
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b
def power(base, exponent):
    """Calculate base raised to exponent"""
    return base ** exponent

# Module-level variables
VERSION = "1.0.0"
AUTHOR = "Python Student"
# This runs when module is imported
print(f"Calculator module v{VERSION} loaded")
```

Creating Custom Modules

- string_utils.py

```
""" String utility functions """
def reverse_string(text):
    """Reverse a string"""
    return text[::-1]
def count_words(text):
    """Count words in text"""
    return len(text.split())
def capitalize_words(text):
    """Capitalize each word"""
    return ' '.join(word.capitalize() for word in text.split())
def remove_vowels(text):
    """Remove vowels from text"""
    vowels = "aeiouAEIOU"
    return ''.join(char for char in text if char not in vowels)
```

```
def is_palindrome(text):
    """Check if text is a palindrome"""
    cleaned = ''.join(char.lower() for char in text if
char.isalnum())
    return cleaned == cleaned[::-1]
def word_frequency(text):
    """Count frequency of each word"""
    words = text.lower().split()
    frequency = {}
    for word in words:
        frequency[word] = frequency.get(word, 0) + 1
    return frequency
```

Creating Custom Modules

- main.py - Using Custom Modules

```
""" Main program using custom modules """
# Import custom modules
import calculator
from string_utils import reverse_string, count_words, is_palindrome


def main():
        print("=== Calculator Demo ===")
        print(f"5 + 3 = {calculator.add(5, 3)}")
        print(f"10 - 4 = {calculator.subtract(10, 4)}")
        print(f"6 * 7 = {calculator.multiply(6, 7)}")
        print(f"15 / 3 = {calculator.divide(15, 3)}")
        print(f"2^8 = {calculator.power(2, 8)}")
        print(f"\nModule info: {calculator.VERSION} by
{calculator.AUTHOR}")
        print("\n=== String Utils Demo ===")
        text = "Hello World Python"
        print(f"Original: {text}")
        print(f"Reversed: {reverse_string(text)}")
        print(f"Word count: {count_words(text)}")
        print(f"Is palindrome: {is_palindrome('racecar')}")
        print(f"Is palindrome: {is_palindrome(text)}")
if __name__ == "__main__":
        main()
```

The name == "main" Pattern

- Understanding Module Execution

```
# demo_module.py
print("This always runs when module is imported/executed")
print(f"Module name is: {__name__}")

def greet(name):
    return f"Hello, {name}!"

# This only runs when file is executed directly
if __name__ == "__main__":
    print("This module is being run directly")
    print(greet("World"))

    # Test functions
    print("Running tests...")
    assert greet("Alice") == "Hello, Alice!"
    print("All tests passed!")
else:
    print("This module is being imported")
```

This always runs when module is imported/executed
Module name is: __main__
This module is being run directly
Hello, World!
Running tests...
All tests passed!

This always runs when module is imported/executed
Module name is: demo_module
This module is being imported
Now using the imported module:

The name == "main" Pattern

- Practical Example

```python
# temperature_converter.py
""" Temperature conversion utilities """
def celsius_to_fahrenheit(celsius):
    """Convert Celsius to Fahrenheit"""
    return celsius * 9/5 + 32
def fahrenheit_to_celsius(fahrenheit):
    """Convert Fahrenheit to Celsius"""
    return (fahrenheit - 32) * 5/9
def celsius_to_kelvin(celsius):
    """Convert Celsius to Kelvin"""
    return celsius + 273.15
def kelvin_to_celsius(kelvin):
    """Convert Kelvin to Celsius"""
    return kelvin - 273.15
```

```python
# Interactive CLI when run directly
if __name__ == "__main__":
    print("Temperature Converter: 1. Celsius to Fahrenheit 2. Fahrenheit to Celsius 3. Celsius to Kelvin 4. Kelvin to Celsius ")
    try:
        choice = int(input("Choose conversion (1-4): "))
        temp = float(input("Enter temperature: "))
        if choice == 1:
            result = celsius_to_fahrenheit(temp)
            print(f"{temp}°C = {result:.2f}°F")
        elif choice == 2:
            result = fahrenheit_to_celsius(temp)
            print(f"{temp}°F = {result:.2f}°C")
        elif choice == 3:
            result = celsius_to_kelvin(temp)
            print(f"{temp}°C = {result:.2f}K")
        elif choice == 4:
            result = kelvin_to_celsius(temp)
            print(f"{temp}K = {result:.2f}°C")
        else: print("Invalid choice!")
    except ValueError:
        print("Please enter valid numbers!")
```

What is a Package (套件)?

- A package is a directory containing multiple modules
- Package Structure

```
my_package/
├── __init__.py              # Makes it a package
├── math_tools/
│   ├── __init__.py
│   ├── basic.py             # Basic math operations
│   ├── advanced.py          # Advanced math functions
│   └── statistics.py        # Statistical functions
├── string_tools/
│   ├── __init__.py
│   ├── formatting.py        # String formatting
│   └── validation.py        # String validation
└── file_tools/
    ├── __init__.py
    ├── readers.py           # File reading utilities
    └── writers.py           # File writing utilities
```

Creating init.py Files

- my_package/init.py

```
""" My Package - A collection of useful utilities """
__version__ = "1.0.0"
__author__ = "Python Student"

# Import commonly used functions 導入常用函數
from .math_tools.basic import add, subtract, multiply, divide
from .string_tools.formatting import capitalize_all, snake_to_camel

# Package-level constants 套件層級常數
PACKAGE_NAME = "My Utilities Package"

print(f"Loaded {PACKAGE_NAME} v{__version__}")
```

Creating init.py Files

- my_package/math_tools/init.py

```
""" Math tools subpackage """

# Import all functions from submodules
from .basic import *
from .advanced import *
from .statistics import *

# Define what gets imported with "from math_tools import *"
__all__ = [
        'add', 'subtract', 'multiply', 'divide',        # from basic
        'power', 'sqrt', 'factorial',                   # from advanced
        'mean', 'median', 'mode'                        # from statistics
]
```

Creating init.py Files

- my_package/math_tools/basic.py

```
""" Basic mathematical operations """


def add(a, b):
        """Add two numbers"""
        return a + b
def subtract(a, b):
        """Subtract two numbers"""
        return a - b
def multiply(a, b):
        """Multiply two numbers"""
        return a * b
def divide(a, b):
        """Divide two numbers"""
        if b == 0:
                raise ValueError("Cannot divide by zero")
        return a / b
```

Creating init.py Files

- my_package/math_tools/advanced.py

```
""" Advanced mathematical functions """


import math
def power(base, exponent):
        """Calculate base raised to exponent"""
        return base ** exponent
def sqrt(number):
        """Calculate square root"""
        if number < 0:
                raise ValueError("Cannot calculate square root of
negative number")
        return math.sqrt(number)
def factorial(n):
        """Calculate factorial recursively"""
        if n < 0:
                raise ValueError("Factorial not defined for negative
numbers")
        if n <= 1:
                return 1
        return n * factorial(n - 1)
```

```
def gcd(a, b):
        """Calculate greatest common divisor"""
        while b:
                a, b = b, a % b
        return a
def lcm(a, b):
        """Calculate least common multiple"""
        return abs(a * b) // gcd(a, b)
```

Creating init.py Files

- Using Packages

```
# Different ways to import from packages
# 1. Import entire package
import my_package
print(my_package.add(5, 3)) # Uses function imported in __init__.py
# 2. Import subpackage
from my_package import math_tools
print(math_tools.factorial(5))
# 3. Import specific module
from my_package.math_tools import advanced
print(advanced.sqrt(16))
# 4. Import specific function
from my_package.math_tools.basic import multiply
print(multiply(4, 7))
# 5. Import with alias
from my_package.math_tools import statistics as stats
data = [1, 2, 3, 4, 5]
print(stats.mean(data))
```

What is Functional Programming?

- Functional programming is a programming paradigm that treats computation as evaluation of mathematical functions
- Key Principles
  - Pure Functions
  - Immutability (不可變性)
  - Higher-Order Functions
  - Function Composition
  - No Side Effects

Pure Functions

- A pure function always produces the same output for the same input and has no side effects

```
# Pure function examples
def add_pure(a, b):
    """Pure function - no side effects, same input = same output"""
    return a + b
def multiply_pure(x, y):
    """Pure function for multiplication"""
    return x * y
def square_pure(n):
    """Pure function to calculate square"""
    return n ** 2
```

```
# Testing pure vs impure
print(add_pure(2, 3)) # Always 5
print(add_pure(2, 3)) # Always 5
print(add_impure(2, 3)) # 5, but with side effects
print(add_impure(2, 3)) # 5, but counter changed
```

```
# Impure function examples
import random
import datetime

counter = 0 # Global variable
def add_impure(a, b):
    """Impure function - modifies global state"""
    global counter
    counter += 1 # Side effect!
    print(f"Function called {counter} times") # Side effect!
    return a + b
def get_random():
    """Impure function - different output each time"""
    return random.randint(1, 100) # Non-deterministic
def get_current_time():
    """Impure function - depends on external state"""
    return datetime.datetime.now() # Depends on system time
```

Benefits of Pure Functions

```python
# 1. Easy to test
def calculate_tax(income, rate):
    """Pure function for tax calculation"""
    return income * rate

# Test is simple and predictable
    assert calculate_tax(1000, 0.1) == 100
    assert calculate_tax(5000, 0.2) == 1000

# 2. Easy to debug
def distance(x1, y1, x2, y2):
    """Calculate distance between two points"""
    return ((x2 - x1) ** 2 + (y2 - y1) ** 2) ** 0.5
# No hidden dependencies or side effects
```

```python
# 3. Cacheable (memoization) 可快取（記憶化）
def memoize(func):
    """Decorator to cache function results"""
    cache = {}
    def wrapper(*args):
        if args in cache:
            print(f"Cache hit for {args}")
            return cache[args]
        result = func(*args)
        cache[args] = result
        print(f"Computed and cached {args} = {result}")
        return result
    return wrapper
@memoize
def expensive_calculation(n):
    """Simulate expensive pure function"""
    print(f"Computing for {n}...")
    result = sum(i ** 2 for i in range(n))
    return result
# Test memoization 測試記憶化
print(expensive_calculation(1000)) # Computed
print(expensive_calculation(1000)) # Cache hit
print(expensive_calculation(500)) # Computed
print(expensive_calculation(1000)) # Cache hit
```

Immutability

- Immutability means that once created, an object cannot be changed

```
# Immutable data types in Python
# int, float, string, tuple, frozenset

# Strings are immutable
original_string = "Hello"
# original_string[0] = "h" # Error! TypeError: 'str' object doesn't support item assignment
# Instead, create new string
new_string = "h" + original_string[1:]
print(f"Original: {original_string}") # Hello
print(f"New: {new_string}") # hello

# Tuples are immutable
original_tuple = (1, 2, 3)
# original_tuple[0] = 10 # Error! TypeError: 'tuple' object doesn't support item assignment
# Instead, create new tuple
new_tuple = (10,) + original_tuple[1:]
print(f"Original: {original_tuple}") # (1, 2, 3)
print(f"New: {new_tuple}") # (10, 2, 3)
```

```
# Working with immutable patterns
def add_item_to_list_mutable(lst, item):
    """Mutable approach - modifies original list"""
    lst.append(item)
    return lst
def add_item_to_list_immutable(lst, item):
    """Immutable approach - returns new list"""
    return lst + [item]

# Comparison
original_list = [1, 2, 3]
print(f"Original list: {original_list}")
# Mutable approach
mutable_result = add_item_to_list_mutable(original_list.copy(), 4)
print(f"Mutable result: {mutable_result}")
# Immutable approach
immutable_result = add_item_to_list_immutable([1, 2, 3], 4)
print(f"Immutable result: {immutable_result}")
print(f"Original unchanged: {[1, 2, 3]}")
```

Function Composition

- Combining Functions

```
# Basic function composition
def add_one(x):
        return x + 1
def multiply_by_two(x):
        return x * 2
def square(x):
        return x ** 2
# Manual composition
def compose_manually(x):
        step1 = add_one(x) # x + 1
        step2 = multiply_by_two(step1) # (x + 1) * 2
        step3 = square(step2) # ((x + 1) * 2)^2
        return step3

print(compose_manually(3))
# ((3 + 1) * 2)^2 = (4 * 2)^2 = 8^2 = 64

# Nested function calls
result = square(multiply_by_two(add_one(3)))
print(result) # 64
```

```
# Generic composition function
def compose(*functions):
        """Compose multiple functions into one"""
        def composed_function(x):
                result = x
                for func in reversed(functions):
                        result = func(result)
                return result
        return composed_function

# Using composition
pipeline = compose(square, multiply_by_two, add_one)
print(pipeline(3)) # 64
```

Function Composition

- Combining Functions

```
# More practical example
def strip_whitespace(text):
        return text.strip()
def convert_to_lowercase(text):
        return text.lower()
def replace_spaces_with_underscores(text):
        return text.replace(' ', '_')
# Create text processing pipeline
process_text = compose( replace_spaces_with_underscores,
convert_to_lowercase, strip_whitespace )
text = " Hello World Python "
processed = process_text(text)
print(f"Original: '{text}' ")
print(f"Processed: '{processed}'") # hello_world_python
```

What are Decorators (裝飾器)?

- Decorators are a powerful way to modify or enhance functions without changing their code

```
# Step 1: Define the Decorator
def my_decorator(func): # func is the function being decorated
    def wrapper(*args, **kwargs):  # wrapper is the new function
        print(f"Before calling {func.__name__}")
        result = func(*args, **kwargs) # Call the original function
        print(f"After calling {func.__name__}")
        return result
    return wrapper # Return the new function
# Step 2: Apply the Decorator
# The @ syntax is equivalent to: greet = my_decorator(greet)
@my_decorator
def greet(name):
    print(f"Hello, {name}!")
    return f"Greeting for {name}"
# Call decorated function
result = greet("Alice")
print(f"Result: {result}")
```

What are Decorators (裝飾器)?

```python
# Timing decorator
import time
import functools
def timing_decorator(func):
    """Decorator to measure function execution time"""“
    @functools.wraps(func) # Preserves original function metadata
    def wrapper(*args, **kwargs):
        start_time = time.time()
        result = func(*args, **kwargs)
        end_time = time.time()
        execution_time = end_time - start_time
        print(f"{func.__name__} executed in {execution_time:.4f} seconds")
        return result
    return wrapper
```

```python
@timing_decorator
def slow_function():
    """Simulate a slow function"""
    time.sleep(1)
    return "Done!"
@timing_decorator
def fibonacci(n):
    """Calculate fibonacci number (inefficient version)"""
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

# Test timing decorator
result = slow_function()
print(f"Result: {result}")

# fib_result = fibonacci(10) # This will show timing for each recursive call
```

What are Iterators (迭代器)?

- An iterator is an object that implements the iterator protocol with __iter__() and __next__() methods
- Iterator Protocol

```
# Any object that implements these two methods is an
iterator


class MyIterator:
    def __iter__(self):
        """Returns the iterator object itself"""
        return self
    def __next__(self):
        """Returns the next value or raises StopIteration"""
        # Logic to return next value
        pass
```

Understanding Iterators (迭代器)

- Iterables vs Iterators

```
# Iterable: An object that can return an iterator
my_list = [1, 2, 3, 4, 5] # This is an iterable
# Get iterator from iterable
my_iterator = iter(my_list) # This is an iterator
# Use iterator
print(next(my_iterator)) # 1
print(next(my_iterator)) # 2
print(next(my_iterator)) # 3
```

```
# Behind the scenes of a for loop
# for item in my_list:
#     print(item)
#
# Is actually doing:
# iterator = iter(my_list)
# while True:
#     try:
#         item = next(iterator)
#         print(item)
#     except StopIteration:
#         break
```

Understanding Iterators (迭代器)

- Common Built-in Iterators

```
# 1. Lists, tuples, strings are iterables
numbers = [1, 2, 3]
text = "Hello"

for num in numbers: # Works!
    print(num)
for char in text: # Works!
    print(char)

# 2. Range objects
for i in range(5):
    print(i) # 0, 1, 2, 3, 4
```

```
# 3. File objects
# with open('file.txt', 'r') as file:
#    for line in file: # File is an iterator!
#        print(line)

# 4. Dictionary views
my_dict = {'a': 1, 'b': 2, 'c': 3}
for key in my_dict.keys(): # Iterate over keys
    print(key)
for value in my_dict.values(): # Iterate over values
    print(value)
for item in my_dict.items(): # Iterate over key-value pairs
    print(item)
```

Creating Custom Iterators (迭代器)

- Simple Counter Iterator

```
class Counter:
    """Iterator that counts from start to end"""
    def __init__(self, start, end):
        self.current = start
        self.end = end
    def __iter__(self):
        """Return the iterator object (self)"""
        return self
    def __next__(self):
        """Return the next value"""
        if self.current >= self.end:
            raise StopIteration
            # Signal that iteration is complete
        value = self.current
        self.current += 1
        return value
```

```
# Using the custom iterator
counter = Counter(1, 6)
for num in counter:
    print(num) # Prints: 1, 2, 3, 4, 5

# Manual iteration
counter2 = Counter(10, 13)
print(next(counter2)) # 10
print(next(counter2)) # 11
print(next(counter2)) # 12
# print(next(counter2)) # Raises StopIteration
```

What are Generators (產生器)?

- Generators are functions that return an iterator object which produces values on-demand

```python
# Generator function using yield
def simple_generator():
    """Simple generator that yields three values"""
    print("Starting generator")
    yield 1
    print("After first yield")
    yield 2
    print("After second yield")
    yield 3
    print("Generator finished")
```

```python
# Using the generator
gen = simple_generator()
print(f"Generator object: {gen}") # Generator object: <generator object simple_generator at 0x...>

# Get values one by one 逐一取得值
print(f"First value: {next(gen)}") # Starting generator # First value: 1
print(f"Second value: {next(gen)}")  # After first yield # Second value: 2
print(f"Third value: {next(gen)}") # After second yield # Third value: 3
# print(f"Fourth value: {next(gen)}")
# StopIteration exception!

# Using generator in loop
print("\n=== Using in loop ===")
for value in simple_generator():
    print(f"Loop value: {value}")
```

What are Generators (產生器)?

- Practical Generator Examples

```python
# Fibonacci generator
def fibonacci_generator(max_count=None):
    """Generate Fibonacci sequence"""
    a, b = 0, 1
    count = 0
    while max_count is None or count < max_count:
        yield a
        a, b = b, a + b
        count += 1
# Generate first 10 Fibonacci numbers
print("First 10 Fibonacci numbers:")
for i, fib in enumerate(fibonacci_generator(10)):
    print(f"F({i}) = {fib}")
```

```python
# Prime number generator
def prime_generator(max_num=None):
    """Generate prime numbers"""
    def is_prime(n):
        if n < 2:
            return False
        for i in range(2, int(n**0.5) + 1):
            if n % i == 0:
                return False
        return True
    num = 2
    while max_num is None or num <= max_num:
        if is_prime(num):
            yield num
        num += 1
# Generate primes up to 50
print("\nPrime numbers up to 50:")
primes = list(prime_generator(50))
print(primes)
```

What are Generators (產生器)?

- Practical Generator Examples

```
# File line reader generator
def read_lines(filename):
    """Generator to read file lines one by one"""
    try:
        with open(filename, 'r', encoding='utf-8') as file:
            for line_num, line in enumerate(file, 1):
                yield line_num, line.strip()
    except FileNotFoundError:
        print(f"File {filename} not found")
        return
```

```
# Range generator (like built-in range)
def my_range(start, stop=None, step=1):
    """Custom range generator"""
    if stop is None:
        start, stop = 0,
    start current = start
    if step > 0:
        while current < stop:
            yield current
            current += step
    elif step < 0:
        while current > stop:
            yield current
            current += step
    else:
        raise ValueError("Step cannot be zero")
# Test custom range
print("\nCustom range(5, 15, 2):")
for num in my_range(5, 15, 2):
    print(num, end=' ‘)
print()
```